

MATH00041: Introduction to Programming

Control Flow

Lecture 4

Archontis Giannakidis, PhD

Outline

- **If/IF-else/If-elif-else**
- **While**
- **Loop**

Python Conditions and If statements

The if statement is a decision-making statement used to determine whether a certain statement or block of statements will execute. Python provides support for the regular logical conditions used in mathematics:

- Equals: `a == b`
- Not Equals: `a != b`
- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`

These conditions can be used in several ways, commonly in "if statements" and loops.

Syntax:

```
if condition:  
    # Statements to execute if  
    # condition is true
```

In Python, a condition is evaluated as either true or false. If the statement accepts boolean values, it will execute the block of statements below it if the value is true, and skip it otherwise.

To identify a block in Python, we use indentation. The block under an if statement will be identified using indentation, as shown in the example below.

An "if statement" is written by using the if keyword.

Example:

If statement:

```
a = 33
```

```
b = 200
```

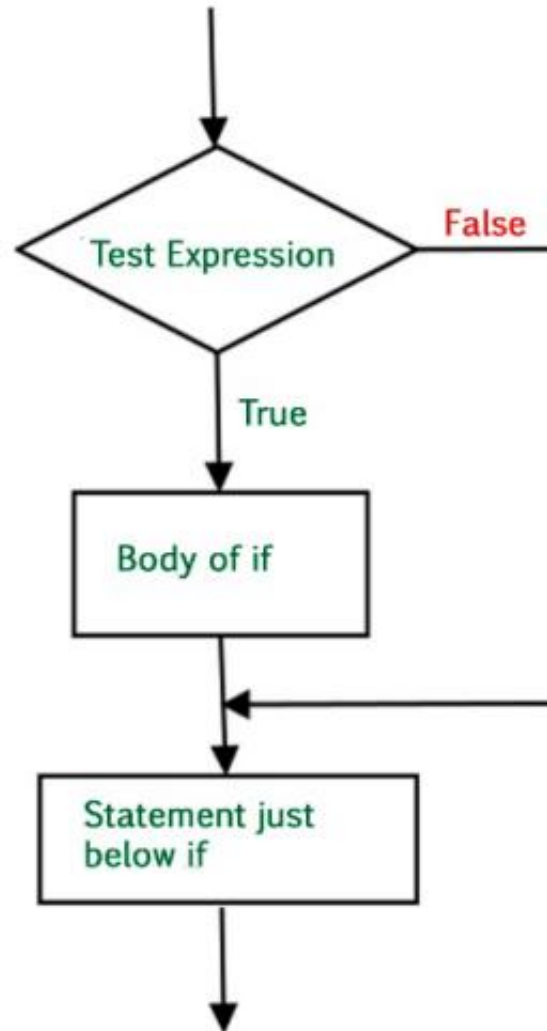
```
if b > a:
```

```
    print("b is greater than a")
```

Output: "b is greater than a"

In this example, we have two variables: 'a' and 'b'. We use them as part of an 'if' statement to test whether 'b' is greater than 'a'. Since 'a' has a value of 33 and 'b' has a value of 200, we know that 200 is greater than 33. Therefore, we print the message "b is greater than a" on the screen.

Flowchart of Python if statement



Else

The if statement is a conditional statement that executes a block of code only if the condition specified is true. On the other hand, if we want to execute a different block of code when the condition is false, we use the else statement with the if statement. The else statement allows us to specify a block of code that is executed only if the if condition is false.

Syntax:

```
if (condition):  
    # Executes this block if  
    # condition is true  
else:  
    # Executes this block if  
    # condition is false
```

The else keyword catches anything which isn't caught by the preceding conditions.

Example:

```
a = 200
```

```
b = 33
```

```
if b > a:
```

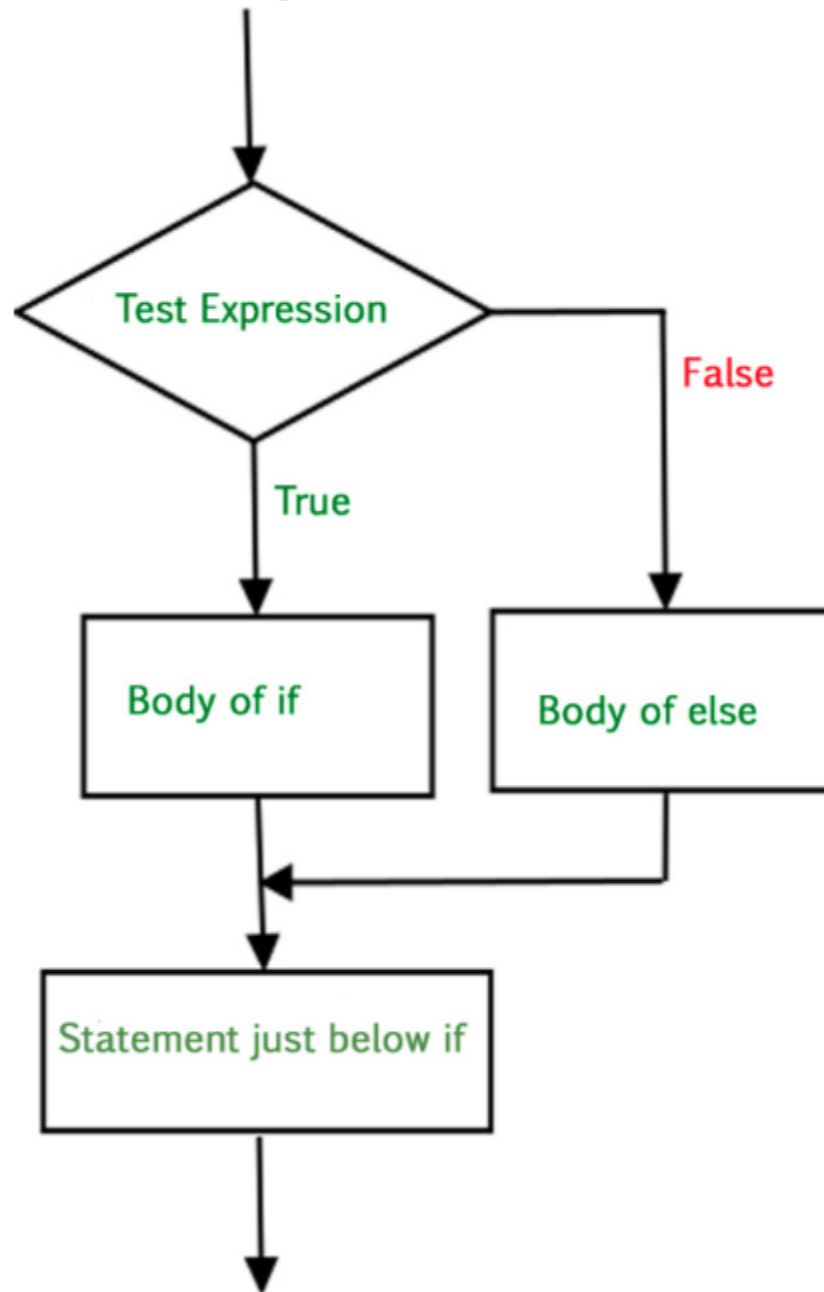
```
    print("b is greater than a")
```

```
else:
```

```
    print("b is not greater than a")
```

Output: "b is not greater than a"

Flowchart of Python if-else statement



Elif

When there are multiple options, a user can choose among them using an `if-elif-else` statement in Python. The `if` statement is executed first, and if its condition is true, the code block associated with that `if` statement is executed and the rest of the code is skipped. If the `if` statement's condition is false, the `elif` statement is checked. If its condition is true, the code block associated with that `elif` statement is executed and the rest of the code is skipped. The final `else` statement will be executed if none of the conditions is true.

Syntax:

```
if (condition):  
    statement  
elif (condition):  
    statement  
.  
.  
else:  
    statement
```

The 'elif' keyword in Python is used when the previous conditions are not true, and the program needs to try another condition.

Example:

```
a = 33
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
```

Output: "a and b are equal"

In this example a is equal to b, so the first condition is not true, but the elif condition is true, so we print to screen that "a and b are equal".

The else keyword catches anything which isn't caught by the preceding conditions.

Example:

```
a = 200
```

```
b = 33
```

```
if b > a:
```

```
    print("b is greater than a")
```

```
elif a == b:
```

```
    print("a and b are equal")
```

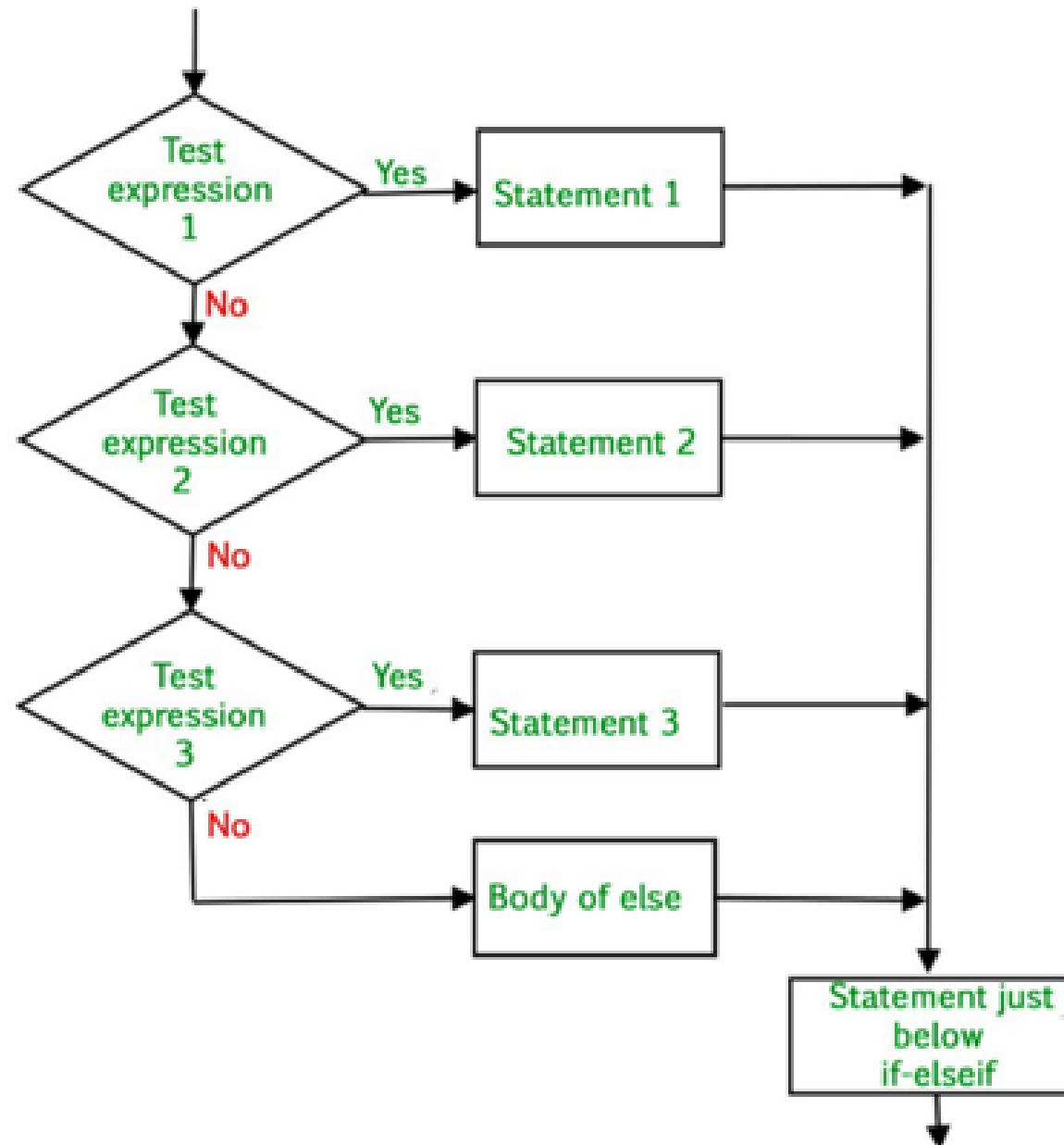
```
else:
```

```
    print("a is greater than b")
```

Output: "a is greater than b"

In this example, if a is greater than b, the else condition is executed and "a is greater than b" is printed to the screen.

Flowchart of Python if-elif-else ladder



Shorthand If

If you have only one statement to execute, you can put it on the same line as the if statement.

Example:

One line if statement:

```
a = 200
```

```
b = 33
```

```
if a > b: print("a is greater than b")
```

Output: "a is greater than b"

Shorthand If ... Else

If you have only one statement to execute, one for if, and one for else, you can put it all on the same line:

Example:

One line if else statement:

```
a = 2  
b = 330  
print("A") if a > b else print("B")
```

Output: "B"

You can also have multiple else statements on the same line:

Example:

One line if else statement:

```
a = 330  
b = 330  
print("A") if a > b else print("=") if a == b else print("B")
```

Output: "="

And

The 'and' keyword is a logical operator that is utilized to combine conditional statements.

Example:

Test if **a** is greater than **b**, AND if **c** is greater than **a**:

```
a = 200
```

```
b = 33
```

```
c = 500
```

```
if a > b and c > a:
```

```
    print("Both conditions are True")
```

Output: "Both conditions are True"

Or

The "or" keyword is a logical operator that is utilized to join conditional statements.

Example:

Test if `a` is greater than `b`, OR if `a` is greater than `c`:

```
a = 200
```

```
b = 33
```

```
c = 500
```

```
if a > b or a > c:
```

```
    print("At least one of the conditions is True")
```

Output: "At least one of the conditions is True"

Not

The 'not' keyword is a logical operator used to negate a conditional statement.:

Example:

Test if **a** is NOT greater than **b**:

```
a = 33
```

```
b = 200
```

```
if not a > b:
```

```
    print("a is NOT greater than b")
```

Output: "a is NOT greater than b"

Nested If

If statements can be nested within one another, creating what is known as nested if statements.

Example:

```
x = 41
```

```
if x > 10:  
    print("Above ten,")  
    if x > 20:  
        print("and also above 20!")  
    else:  
        print("but not above 20.")
```

Output: Above ten,

Output: and also above 20!

The pass Statement.

if statements cannot be empty, but if you for some reason have an if statement with no content, put in the pass statement to avoid getting an error.

Example:

```
a = 33
```

```
b = 200
```

```
if b > a:
```

```
    pass # having an empty if statement like this, would raise an error  
without the pass statement
```

Output:

Python Loops

Python provides two fundamental loop commands:

- while loops
- for loops

The while Loop

Python's While Loop executes a block of statements repeatedly until a given condition is met. Once the condition is false, the following line is executed.

Syntax:

```
while expression:  
    statement(s)
```

The while loop is a type of indefinite iteration, meaning that the number of times the loop will execute is not specified in advance.

In Python, a block of code is defined by the statements that are indented by the same number of spaces after a programming construct. The while loop uses indentation to group its statements. When a while loop is executed, the expression is first evaluated in a Boolean context. If it is true, the loop body is executed. The expression is then checked again, and if it is still true, the loop body is executed again. This process continues until the expression becomes false.

Example:

Print i as long as i is less than 6:

```
i = 1
while i < 6:
    print(i)
    i += 1
```

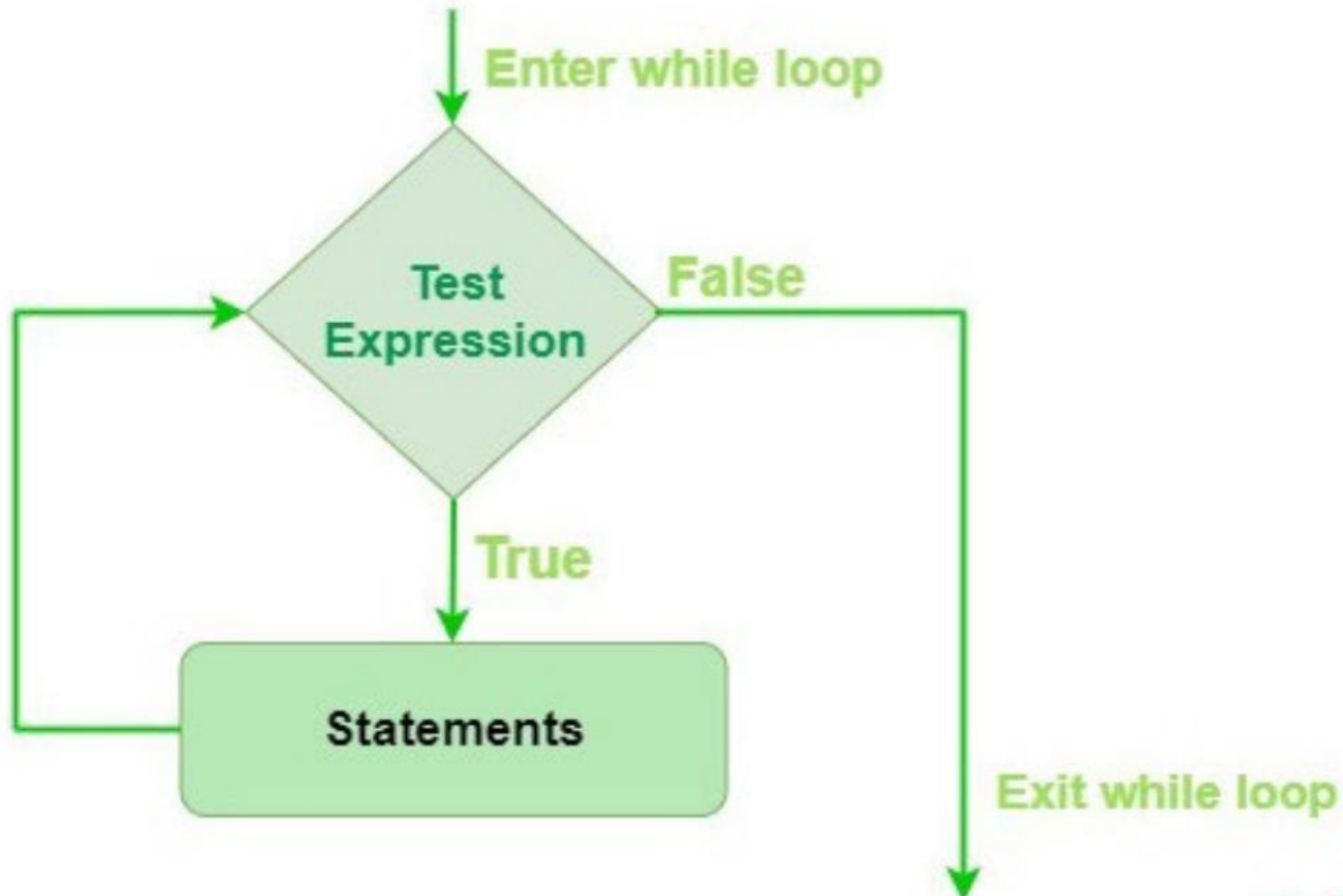
Output:

1
2
3
4
5

❖ Note: remember to increment i, or else the loop will continue forever.

The while loop requires relevant variables to be ready. In this example, we define an indexing variable i and set it to 1.

Flowchart of While Loop



The break Statement

Using the break statement within a while loop allows for early termination of the loop, even if the while condition remains true.

Example:

Exit the loop when i is 3:

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

Output:

1
2
3

The continue Statement

Using the continue statement allows us to skip the current iteration and move on to the next one.

Example:

Continue to the next iteration if i is 3:

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

Output:

1
2
4
5
6

The else Statement

Using the else statement, we can execute a block of code when the initial condition is false:

Example:

Print a message once the condition is false:

```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```

Output:

1
2
3
4
5

i is no longer less than 6

Iterating Over Tuple Using While Loops

You can use a while loop to loop through the items of a tuple. First, determine the length of the tuple using the `len()` function. Starting at index 0, you can refer to the items of the tuple using their respective indexes. After each iteration, remember to increment the index by 1 to move to the next item.

Example:

Print all items, using a while loop to go through all the index numbers:

```
thistuple = ("apple", "banana", "cherry")
i = 0
while i < len(thistuple):
    print(thistuple[i])
    i = i + 1
```

Output: apple

Output: banana

Output: cherry

Iterating Over Lists Using While Loops

To loop through the list items, you can use a **while** loop. First, determine the length of the list using the **len()** function. Then, start at index 0 and use a loop to iterate through each item by referring to their corresponding indexes. After each iteration, increase the index by 1.

Example:

Print all items, using a **while** loop to go through all the index numbers:

```
thislist = ["apple", "banana", "cherry"]  
i = 0  
while i < len(thislist):  
    print(thislist[i])  
    i = i + 1
```

Output: apple

Output: banana

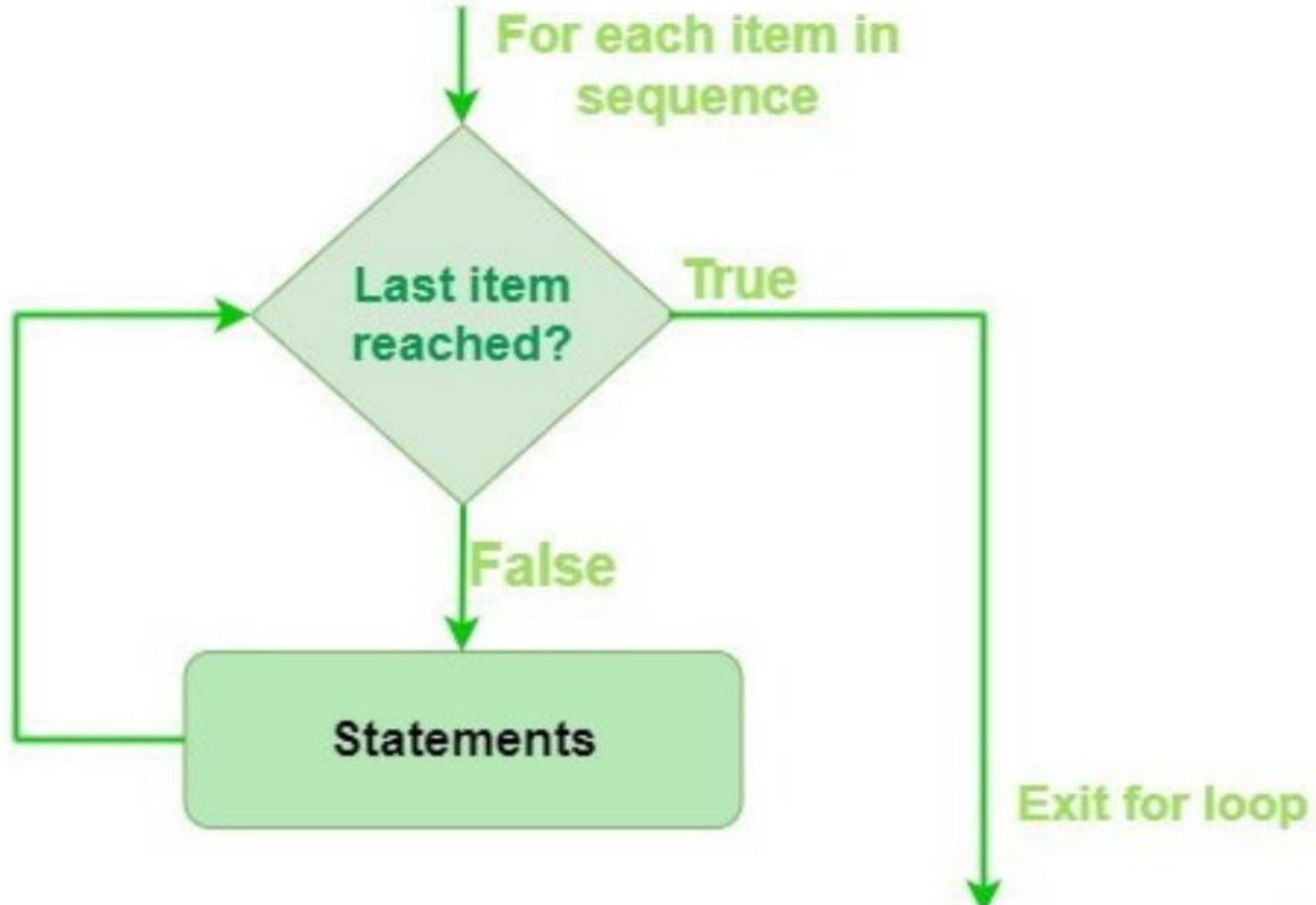
Output: cherry

Python For Loops

A for loop is a programming construct used for iterating over a sequence. The sequence can be a list, a tuple, a dictionary, a set, or a string. Unlike the for loop in other programming languages, it functions more like an iterator method found in object-oriented programming languages.

The for loop allows us to execute a group of statements, once for every item in a list, tuple, set, or any other sequence.

Flowchart of for loop



Example:

Print each fruit in a fruit list:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

Output:

apple

banana

cherry

You don't need to set an indexing variable beforehand in a for loop.

Looping Through a String

Although strings are not usually thought of as iterable objects, they are in fact just that. A string is a sequence of characters and can be iterated through just like any other iterable object.

Example:

Loop through the letters in the word "banana":

```
for x in "banana":  
    print(x)
```

Output:

b
a
n
a
n
a

The break Statement

By using the break statement, we can prematurely end the loop.

Example:

Exit the loop when `x` is "banana":

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)  
    if x == "banana":  
        break
```

Output:

apple

banana

The continue Statement

By using the continue statement, we can skip the current iteration of a loop and proceed with the next one.

Example:

Do not print banana:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana":  
        continue  
    print(x)
```

Output:

apple

cherry

The range() Function

To loop through a set of code a specified number of times, we can use the range() function,

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Example:

Using the range() function:

```
for x in range(6):  
    print(x)
```

Output:

```
0  
1  
2  
3  
4  
5
```

❖ Note: that range(6) is not the values of 0 to 6, but the values 0 to 5.

The range() function starts from 0, but can be customized with a starting value. For example, range(2, 6) produces values from 2 to 5.

Example:

Increment the sequence with 3 (default is 1):

```
for x in range(2, 30, 3):  
    print(x)
```

Output:

2

5

8

11

14

17

20

23

26

29

Else in For Loop

The else statement in a for loop defines a block of code to be executed if the loop finishes normally:

Example:

Print all numbers from 0 to 5, and print a message when the loop has ended:

```
for x in range(6):  
    print(x)  
else:  
    print("Finally finished!")
```

Output:

0

1

2

3

4

5

Finally finished!

❖ Note: The else block will NOT be executed if a break statement stops the loop.

Nested Loops

A nested loop is a loop inside another loop. The inner loop executes once for each iteration of the outer loop.

Example:

Print each adjective for every fruit:

```
adj = ["red", "big", "tasty"]  
fruits = ["apple", "banana", "cherry"]
```

```
for x in adj:  
    for y in fruits:  
        print(x, y)
```

Output:

```
red apple  
red banana  
red cherry  
big apple  
big banana  
big cherry  
tasty apple  
tasty banana  
tasty cherry
```

The pass Statement

for loops cannot be empty, but if you have a for loop with no content, put in the pass statement to avoid getting an error.

Example:

```
for x in [0, 1, 2]:  
    pass # having an empty for loop like this, would raise an error  
without the pass statement
```

Output:

Loop Through a Tuple

You can loop through the tuple items by using a **for** loop.

Example:

Iterate through the items and print the values:

```
thistuple = ("apple", "banana", "cherry")  
for x in thistuple:  
    print(x)
```

Output: apple

Output: banana

Output: cherry

Loop Through the Index Numbers

You can also loop through the tuple items by referring to their index number.

Use the `range()` and `len()` functions to create a suitable iterable.

Example:

Print all items by referring to their index number:

```
thistuple = ("apple", "banana", "cherry")  
for i in range(len(thistuple)):  
    print(thistuple[i])
```

Output: apple

Output: banana

Output: cherry

Loop Through a List

You can loop through the list items by using a **for** loop:

Example:

Print all items in the list, one by one:

```
thislist = ["apple", "banana", "cherry"]  
for x in thislist:  
    print(x)
```

Output: apple

Output: banana

Output: cherry

Loop Through the Index Numbers

You can also loop through the list items by referring to their index number.

Use the `range()` and `len()` functions to create a suitable iterable.

Example:

Print all items by referring to their index number:

```
thislist = ["apple", "banana", "cherry"]  
for i in range(len(thislist)):  
    print(thislist[i])
```

Output: apple

Output: banana

Output: cherry

❖ Note : The iterable created in the example above is [0, 1, 2].

Comprehension

The Syntax:

```
newlist = [expression for item in iterable if condition == True]
```

❖ Note: The return value is new, leaving the old list unchanged.

comprehension provides a concise way to create a new list based on the values of an existing list. For instance, if you have a list of fruits and you want to create a new list that only contains the fruits with the letter "a" in their name, you will have to use a **for** loop with a conditional test inside, if not using list comprehension.

Example:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = []
```

```
for x in fruits:  
    if "a" in x:  
        newlist.append(x)
```

```
print(newlist)
```

Output: ['apple', 'banana', 'mango']

With list comprehension, you can do all that with only one line of code:

Example:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
```

```
newlist = [x for x in fruits if "a" in x]
```

```
print(newlist)
```

Output: ['apple', 'banana', 'mango']

Conditional Comprehension

The condition acts like a filter that only allows the items that evaluate to True.

Example:

Only accept items that are not "apple":

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = [x for x in fruits if x != "apple"]
```

Output: ['banana', 'cherry', 'kiwi', 'mango']

The condition `if x != "apple"` will return `True` for all elements other than `"apple"`, making the new list contain all fruits except `"apple"`.

Iterable Comprehension

The iterable can be any object that can be iterated over, such as a list, tuple, or set.

Example:

You can use the `range()` function to create an iterable:

```
newlist = [x for x in range(10)]
```

Output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Same example, but with a condition:

Example:

Accept only numbers lower than 5:

```
newlist = [x for x in range(10) if x < 5]
```

Output: [0, 1, 2, 3, 4]

Expression Comprehension

The expression is the current item in the iteration, but it is also the outcome, which you can manipulate before it ends up like a list item in the new list:

Example:

Set the values in the new list to upper case:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
```

```
newlist = [x.upper() for x in fruits]
```

Output: ['APPLE', 'BANANA', 'CHERRY', 'KIWI', 'MANGO']

You can set the outcome to whatever you like:

Example:

Set all values in the new list to 'hello':

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = ['hello' for x in fruits]
```

Output: ['hello', 'hello', 'hello', 'hello', 'hello']

The expression can also contain conditions, not like a filter, but as a way to manipulate the outcome:

Example:

Return "orange" instead of "banana":

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
  
newlist = [x if x != "banana" else "orange" for x in fruits]
```

Output: ['apple', 'orange', 'cherry', 'kiwi', 'mango']

The expression above returns the item if it's not banana, otherwise orange.

Thank you